

로드맵 - 생각정리 및 업데이트.RE

1. 로드맵 SKILL 단계 학습 로직 — 확정

배경: 로드맵의 SKILL 타입 단계(부족한 기술을 채우는 단계)가 지금까지는 "완료 체크" 버튼 하나뿐이었음. 여기에 실제로 배웠는지 확인하는 절차가 없었음.

1-1. Tier별 학습 방식 (AI 활용 버전, 최초 확정안)

Tier	하는 일	완료 방식	왜 이렇게 나왔나
ENTRY(입문)	공부노트 작성	제출 → AI가 핵심 개념·코드 예제 포함 여부 체크	배움의 시작 단계는 "이해했는지 확인"이 목적. 정답을 맞히는 게 아니라 성의 있게 공부했는지를 보는 수준으로 기준을 느슨하게 잡음
CORE/ADVANCED(중급)	기존 프로젝트 업그레이드 또는 신규 프로젝트	프로젝트 등록 + 증빙 파일, <code>tech_stack</code> 변화로 자동 확인	"배움"에서 "적용"으로 넘어가는 단계. 이미 있는 로드맵 PROJECT 단계 로직을 그대로 재사용할 수 있어 구현 부담이 적음
EXPERT(고급)	이 기술을 설명하는 기술 글 작성	제출 → AI가 개념 설명·예시·대상 독자 고려 여부 체크	"가르칠 수 있어야 진짜 아는 것"이라는 기준. 오픈소스 기여나 배포 URL 같은 대안도 검토했으나, 외부 요인(리뷰어 승인 등)에 완료가 좌우되는 게 단점이라 제외. 입문 단계의 "노트 제출"과 대칭 구조를 이루는 점도 고려

1-2. 스키마 변경 (확정)

테이블	컬럼	타입	이유
ROADMAP_STEP	<code>proof_type</code>	VARCHAR(20)	<code>NOTE</code> / <code>PROJECT_LINK</code> / <code>TEACHING_POST</code> — tier로 자동 결정되지만 기준이 바뀔 수 있어 명시적으로 저장
ROADMAP_STEP	<code>evidence_project_id</code>	BIGINT FK → USER_PROJECTS, NULL	PROJECT_LINK일 때 어느 프로젝트로 완료했는지 연결
ROADMAP_STEP	<code>review_status</code>	VARCHAR(20)	<code>PENDING</code> / <code>PASSED</code> / <code>NEEDS_REVISION</code> — 노트·기술 글 판정 결과 저장
ROADMAP_STEP	<code>review_note</code>	TEXT	판정 근거·피드백
DOCUMENTS	<code>roadmap_step_id</code>	BIGINT FK → ROADMAP_STEP, NULL	공부노트·기술 글을 프로젝트 없이 바로 스킬 단계에 붙이기 위함
USER_PROJECTS	<code>upgraded_from_project_id</code>	BIGINT FK → USER_PROJECTS(자기참조), NULL	"기존 프로젝트 업그레이드"를 선택했을 때 이전 버전 연결. NULL이면 신규 프로젝트

1-3. ⚠️ 아직 답이 없는 것 — CORE와 ADVANCED 구분

지금은 두 tier가 완료 방식이 똑같아서 난이도 차이가 스키마상 드러나지 않음. 제안했던 안: **ADVANCED는 "반드시 업그레이드"만 인정, CORE는 신규/업그레이드 둘 다 허용.** 확정 안 됨.

→ 확정 됨!! 제안했던 안 그대로 하기로 함!

3. 로드맵이 한 번 만들면 고정되는 문제 — 해결 방향 확정

문제 제기: "출업까지 쓰는 프로젝트인데, 유저는 가만히 있어도 직무 트렌드는 계속 바뀐다. 지금 설계(FR-37, 프로필 변경 시에만 재분석)로는 이 변화를 놓친다."

검토했으나 채택 안 한 안: 트렌드 갱신될 때마다 관련 유저 전원의 로드맵을 자동 재생성. → **AI 비용이 유저 수 × 갱신 주기만큼 반복해서 4번 항목의 비용 문제가 그대로 재발**하므로 기각.

확정안 — 변화 감지 + 알림 방식:

- 직무 요구사항이 실제로 바뀌었는지는 **DB 비교로 감지** (비용 0원)

2. 목표 직무로 삼은 유저에게 **배너로 알림**만 띄움 ("요구 기술이 바뀌었어요")

3. **유저가 직접 눌러야** 재분석·재생성 — 여기서만 AI 비용 발생

이 구조를 택한 이유는 "세상이 변하면 여정도 같이 변한다"는 오디세이 컨셉을 살리면서도, 비용은 정말 필요할 때·유저가 원할 때만 쓰기 때문. 정해진 지도가 아니라 향해하며 다시 그리는 지도라는 서사와도 맞음.

3-1. 스키마 변경 (확정)

테이블	컬럼	타입	이유
JOB	<code>requirement_version</code>	INT	필수 기술 목록이 바뀔 때마다 +1. 내용이 같으면 갱신해도 안 올림
GAP_ANALYSIS	<code>job_requirement_version</code>	INT	이 분석이 어느 버전 기준인지 저장. <code>JOB.requirement_version</code> 과 비교해 낡았는지 판단

ROADMAP의 기존 `version` / `is_active` / `is_primary` 와 단계 승계 로직은 그대로 재사용 — 트렌드 반영 재생성도 프로필 변경 재생성과 같은 흐름을 타므로 새 테이블 불필요.

4. "AI 없이 한다면" — 검토한 대안 (참고용, 확정 아님)

이 논의는 두 단계로 좁혀졌습니다. 처음엔 로드맵 시스템 전체를 AI 없이 만드는 방안을 넓게 검토했고, 이어서 "아니 말고 이전 기술 배우는 로직"이라는 요청으로 **3번 항목(스킬 학습 로직)**만으로 범위가 좁혀졌습니다. 아래는 그 순서대로 정리한 것입니다.

4-1. 로드맵 시스템 전체를 AI 없이 (넓은 범위 — 논의만 하고 스킬 학습 로직으로 범위를 좁혔으므로 미채택)

원래(AI)	AI 없이 대안
부족 기술 tier별 순서 정하기	<code>importance</code> (필수/우대)· <code>difficulty_level</code> 순 정렬 규칙
"왜 지금 해야 하는지" 문장 생성	템플릿 문장 (" <code>{직무명}</code> 의 필수 기술입니다. 요구 비율 <code>{비율}%</code> ")
새 요구 기술 설명	목록으로만 표시
직무 추천, 자소서 첨삭	규칙만으로 만들기 어려운 영역이라 범위에서 제외 제안

→ 문장 생성은 템플릿으로 대체

4-2. 스킬 학습 로직 AI 없이 (직접 논의 대상 — 최종 확정)

중급(CORE/ADVANCED)은 원래도 `tech_stack` 비교라 규칙 기반이었으므로 그대로 유지. 바뀌는 건 ENTRY·EXPERT의 판정 방식.

Tier	제출물	자동 판정 규칙(제안)
ENTRY	공부노트	글자 수 300자 이상 + 기술명 2회 이상 등장 + 코드 블록 1개 이상
EXPERT	기술 설명 글	글자 수 800자 이상 + 기술명 3회 이상 + 외부 링크 1개 이상

`ROADMAP_STEP.review_status` / `review_note` 는 그대로 쓰되, 채우는 주체가 **AI** → **규칙 기반 서비스 로직**으로 바뀜.

한계로 짚은 점: 키워드·글자수 기준이라 의미 없는 내용을 채워도 통과할 수 있음(예: 같은 단어 반복). AI 채점이 하던 "진짜 이해했는지" 검증을 포기하는 대신 비용이 0원이 되는 트레이드오프.

보완책으로 제시했으나 미확정:

- 관리자(팀원)가 나중에 훑어보는 검수 화면 추가 — 자동 판정을 1차 필터, 사람이 2차 필터로
- 통과는 시키되 점수 적립은 관리자 승인 후로 미루기

학생 프로젝트 규모라 악용 유인이 적을 것으로 보이지만, **이 방향(규칙 기반 확정) 및 보완책 채택 여부는 아직 사용자 확답 전 상태**입니다.

4-3. EXPERT 증빙을 "블로그처럼 공개"로 확정

배경: EXPERT 단계의 기술 설명 글을 그냥 서류 보관함에 제출만 하고 끝내는 게 아니라, **서비스 안에서 다른 사용자도 볼 수 있게 공개하자**는 제안이 나옴.

왜 좋은 방향인가:

- 서류 보관함(DOCUMENTS)에 파묻히지 않고 다른 사용자가 찾아볼 수 있는 콘텐츠 자산이 됨
- "가르칠 수 있어야 진짜 아는 것"이라는 EXPERT 단계 취지가 실제로 남을 가르치는 행위로 완성됨
- 같은 기술을 배우려는 사용자 입장에서 트렌드 사이드바보다 오히려 신뢰가 갈 수 있는 실전 콘텐츠
- 졸업 프로젝트 발표에서 "사용자가 콘텐츠를 만들고 서로 배운다"는 어필 포인트

공개 방식 확정 — 자동 게시 + 사후 관리: 규칙 판정(글자 수·키워드·링크 포함 여부)을 통과하면 곧바로 공개되고, 문제가 있으면 팀원이 나중에 훑어보고 내리는 방식. 수동 승인을 먼저 거치게 하는 안과 큐레이션(추천 글만 노출)하는 안도 검토했으나, 완료 시점과 공개 시점이 같다면 사용자 경험이 나빠져서 기각.

4-3-1. 스키마 변경 (확정)

신설 테이블 TECH_ARTICLE

컬럼	타입	설명
id	BIGINT PK	
roadmap_step_id	BIGINT FK → ROADMAP_STEP	어느 학습 단계에서 나왔는지
user_id	BIGINT FK → USERS	작성자
skill_id	BIGINT FK → SKILL	어떤 기술에 대한 글인지 — 다른 사용자가 이 기준으로 찾아봄
title	VARCHAR(200)	
content	TEXT	
status	VARCHAR(20)	DRAFT / PUBLISHED / HIDDEN
published_at	DATETIME, NULL	
view_count	INT	다른 사용자가 몇 번 봤는지 — 인기 글 정렬 기준

노출 위치: 새 화면을 만들지 않고 트렌드 사이드바(FR-54)나 데이터 인사이트 화면에 "이 기술 관련 글" 목록으로 엮는 방향. skill_id 로 연결되므로 기존 화면 자리를 그대로 재사용 가능.

ERD 뷰어-db-design 반영 완료: ROADMAP_STEP (증빙 방식 컬럼 4개), DOCUMENTS (roadmap_step_id), USER_PROJECTS (upgraded_from_project_id), JOB (requirement_version), GAP_ANALYSIS (job_requirement_version), TECH_ARTICLE (신설) — 여기까지 전부 뷰어에 반영하고 검증(테이블 37개, 관계 50개, 컬럼 244개, 카드 겹침·FK 불일치 0건)을 마쳤습니다.

4-4. 프로젝트 완료 시 받는 것 — 확정

배경: 지금까지 CORE/ADVANCED(프로젝트) 완료는 증빙 파일 업로드만으로 판정됐음. 이걸로는 실제 동작 여부, 학습 맥락, 최소한의 문서화가 하나도 안 남는다는 문제 제기가 있었음.

추가로 받기로 한 것 세 가지

받는 것	왜	완료 판정에 쓰이나
코드 저장소 링크 (repo_url)	파일 스크린샷보다 실제로 열어볼 수 있는 링크가 신뢰도 높음. 면접관 뷰에서도 링크 클릭이 파일 다운로드보다 자연스러움	안 씀 (참고용)
완료 회고 2~3줄 (retrospective)	"업그레이드했다"는 사실만 남기지 않고 무엇을 배우고 해결했는지 기록	안 씀 (참고용)
배포 URL (deploy_url , 선택)	있으면 완성도가 확실히 보이지만, 강제하면 4주 일정에 부담	안 씀, NULL 허용

기술 활용 설명서 — 딥러닝·시맨틱 재료용

배경: "이왕 받는 문서들, 추후 딥러닝이나 시맨틱 자료로 쓸 수 있지 않나"는 제안에서 출발.

프로젝트에 등록한 기술마다 "어떻게 활용했는지"를 문장으로 받는 테이블(PROJECT_TECH_NOTE)을 신설. USER_SKILLS.raw_input 이 짧은 원문 → 표준 스킬을 쌍으로 저장하는 것과 같은 발상이지만, 이건 문장 단위의 실사용 맥락이라 데이터가 훨씬 풍부함.

예: "Redis로 세션 캐싱"과 "Redis를 메시지 큐로 사용"은 같은 기술이지만 활용 맥락이 다름 — 이런 차이를 축적해두면 나중에 임베딩 모델을 이 서비스 도메인에 맞게 보정하는 실제 데이터가 됨.

동의 문제: 명세서 NFR-4("민감 데이터는 본인 동의·본인 선택 공유만")를 지키기 위해 `consent_for_training` 컬럼을 넣음. 체크 안 해도 회고·완료 처리에는 지장 없고, 동의한 것만 나중에 학습 데이터로 뽑아 씀. 나중에 소급 동의를 구하는 건 훨씬 번거로우므로 받을 때부터 동의를 같이 받기로 함.

프로젝트 문서 체크리스트 — "없음" 버튼 방식으로 확정

배경: README·기획서·설계문서·API명세서·테스트결과서·발표자료 등 프로젝트에서 나올 만한 문서를 받되, 전부 강제하면 없는 프로젝트도 억지로 빈 문서를 만들어 올리는 부작용이 생김. → 문서 종류마다 "제출" 또는 "없음(해당 없음)" 버튼을 두는 방식으로 확정.

필수 2개만 예외: README, 실행 화면 스크린샷은 "없음" 버튼을 못 누르게 함.

순위	문서	없음 버튼	이유
1	README / 프로젝트 소개서	❌ 제외 (필수)	다른 사람이 이 프로젝트가 뭔지 알 최소 수단. 면접관 뷰에서 프로젝트 클릭 시 최소한 이걸 보여야 함
2	실행 화면 캡처	❌ 제외 (필수)	저장소 링크만으론 "실제로 돌아가는지"가 안 보임. 최소한의 동작 증빙
3	기획서·요구사항 정의서	✅	작은 프로젝트는 이런 문서 없이 바로 코딩하는 경우가 흔함
4	설계 문서(아키텍처·ERD)	✅	미니 프로젝트엔 과할 수 있음
5	API 명세서	✅	API 서버가 아닌 프로젝트엔 해당 없음
6	테스트 결과서	✅	테스트를 안 짤 프로젝트도 많음
7	발표자료·데모 영상 링크	✅	있으면 강력하지만 매번 강제하면 진입 장벽이 큼

필수를 2개로 제한한 이유: README와 스크린샷은 만드는 데 5분이 안 걸리면서 "이게 실제로 존재하고 동작한다"는 최소 증빙은 확실히 됨. 그 이상 늘리면 학습 로직 자체를 회피하게 될 위험.

스키마 변경 (확정, ERD 뷰어 반영 완료)

USER_PROJECTS에 추가

컬럼	타입	설명
<code>repo_url</code>	VARCHAR(500)	코드 저장소 링크
<code>deploy_url</code>	VARCHAR(500)	배포 주소, 선택
<code>retrospective</code>	TEXT	완료 회고

신설 테이블 PROJECT_TECH_NOTE

컬럼	타입	설명
<code>id</code>	BIGINT PK	
<code>project_id</code>	BIGINT FK → USER_PROJECTS	
<code>skill_id</code>	BIGINT FK → SKILL	
<code>description</code>	TEXT	이 기술을 어떻게 활용했는지
<code>consent_for_training</code>	BOOLEAN	학습 활용 동의, 기본 false

신설 테이블 PROJECT_DOCUMENT_ITEM

컬럼	타입	설명
<code>id</code>	BIGINT PK	
<code>project_id</code>	BIGINT FK → USER_PROJECTS	
<code>doc_type</code>	VARCHAR(30)	README/SCREENSHOT/PLANNING/DESIGN/API_SPEC/TEST_REPORT/PRESENTATION
<code>status</code>	VARCHAR(20)	SUBMITTED/NOT_APPLICABLE

컬럼	타입	설명
<code>document_id</code>	BIGINT FK → DOCUMENTS, NULL	SUBMITTED일 때만 채움 — 실제 파일은 기존 DOCUMENTS 재사용

복합 UNIQUE (`project_id, doc_type`) — 같은 종류 문서를 중복 체크하지 않게.

완료 판정 규칙: `doc_type`이 README, SCREENSHOT인 두 행의 `status`가 둘 다 SUBMITTED여야 로드맵 PROJECT 단계 완료 처리.

ERD 뷰어 반영 결과: 테이블 39개, 관계 54개, 컬럼 257개. FK 불일치·카드 겹침·JS 문법 전부 재검증 통과.

5. 프로젝트 문서 색인 — 한눈에 보기

지금까지 만든 문서 전부를 모았습니다. 팀 노션이나 저장소의 `docs/`에 이 순서대로 넣으시면 됩니다.

#	문서	무엇이 들어있나	언제 보면 되나
1	요구사항 명세서 (<code>requirements.md</code>)	FR-11~115, NFR-1~9, TD-1~5. 모든 기능·컬럼의 근거가 되는 원본	기능 하나가 "왜 이렇게 동작해야 하는지" 궁금할 때
2	DB 설계 및 ERD (<code>db-design.md</code> / ERD 뷰어)	테이블 39개 전체 정의, 관계도, 복합 UNIQUE 목록, 설계 판단 근거	컬럼명·타입·제약을 확인할 때. 뷰어는 클릭하며 검수할 때
3	DAO·DTO 가이드 (<code>dao-guide.md</code> / 웹뷰어)	공용 뼈대(TransactionUtil 등) 사용법, DAO 메서드 전체 레퍼런스, 모르고 쓰면 버그 되는 특이 사항	서비스 코드를 짤 때, "이 DAO 메서드 어떻게 쓰지" 싶을 때
4	2주차 역할 분담 (<code>role-assignment-week2.md</code>)	역할 5개, 담당 영역, 월요일 계약(인터페이스), 일정, 완료 기준	이번 주 누가 뭘 하는지, 어디까지 하면 끝인지 확인할 때
5	결정사항 정리 (이 문서, <code>decisions-2026-09-30.md</code>)	화면 설계 산출물, 랭크 아이콘, 스킬 학습 로직, AI 비용 분석, 로드맵 최신화, 프로젝트 완료 증빙 — 오늘 논의한 모든 결정과 그 이유	"이거 왜 이렇게 정했더라" 싶을 때. 체크리스트(7절)에 아직 안 정해진 것도 여기 모여 있음
6	화면 설계 아티팩트	23개 화면 (스타일가이드~D-day), Figma 가져오기용 HTML, PDF 병합본, 스크롤형 뷰어	화면 하나 만들 때 참고하거나, 팀 검수할 때
7	CLAUDE.md	Tomcat 10+/Jakarta 전환 주의사항, 명명 규칙, DB·코드·보안 규칙, 하지 말 것	Claude Code로 작업 시작하기 전에 항상

아직 없는 문서: API 명세서(URL별 요청/응답 형식), 테스트 계획·결과 보고서, 사용자 매뉴얼, 최종 보고서. 2주차 진행하면서 필요해지면 그때 만들면 됩니다.